AMAZON SALES DATA ANALYSIS AND ORDER STATUS PREDICTION USING RANDOM FOREST CLASSIFICATION

U21ADP05 - Exploratory Data Analysis and Visualization

REPORT

Submitted by

BHAVISHA S (24CS031)

DEVAMITHRA A (24CS045)

DIPIKA JASMINE J (24CS057)

SIVARANJINI N (24CS204)

TEJASWINI S L (24CS225)

In partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous Institution)

ARASUR, COIMBATORE 641 407

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "AMAZON SALES DATA ANALYSIS AND ORDER STATUS PREDICTION USING RANDOM FOREST CLASSIFICATION" is the bonafide work of "**Bhavisha S (24CS031), Devamithra A(24CS045), Dipika Jasmine J(24CS057), Sivaranjini N (24CS204), Tejaswini S L (24CS225)** who carried out the project work under the supervision of the faculty-in-charge.

FACULTY IN-CHARGE
Mr. Prem Kumar D
Assistant Professor III
Department of Computer Science and Engineering
KPR Institute of Engineering and Technology,
Coimbatore

Submitted for Project Viva-Voce held on _____

ACKNOWLEDGEMENT

We wish to express our heartfelt thanks and gratitude to our honorable Chairman **Dr. K. P.Ramasamy,** KPR Group of Institutions, for providing the facilities during the course of our study in the college.

We would like to express our deep gratitude and sincere regards to our beloved and honourable Principal **Dr. Akila Muthuramalingam**, KPR Institute of Engineering and Technology, who gave us the opportunity to frame the project to the full satisfaction.

We owe a genuine gratitude to **Dr. Miruna Joe Amali S**, Professor and Head of the Department, Department of Computer Science and Engineering, for her valuable support and encouragement during this project work.

We are grateful to **Mr. Prem Kumar D**, Assistant Professor III, Department of Computer Science and Engineering, the project supervisor, for his timely suggestions and constant encouragement and support that led to the accomplishment of the project.

We owe special thanks to our Year Head **Dr. Priya V**, Associate Professor, Department of Computer Science and Engineering, for her continuous support and guidance throughout the project.

The acknowledgement would be incomplete without a word of thanks to all our parents, faculty members, supporting staff and friends for their continuous support and sincere help throughout our project.

ABSTRACT

This project presents an exploratory analysis and machine learning workflow for an Amazon sales dataset, with the objective of understanding sales patterns and predicting order status. The data are structured transactional records containing order identifiers, date, product, category, price, quantity, total sales, customer information, payment method and status. The notebook loads the dataset in Google Colab and examines its dimensions, data types, descriptive statistics, distributions and numerical relationships.

The dataset contains 250 records and the initial table includes 22 columns in the notebook's prepared state. The analysis uses Pandas and NumPy for data handling, Matplotlib and Seaborn for static visualisation, and Plotly for interactive charts. Exploratory outputs include price histograms, category charts, treemaps, bubble plots, boxplots, violin plots, correlation heatmaps, 3D scatter plots and parallel-coordinate visualisations. Preprocessing examples include mean imputation, label encoding, scaling, outlier inspection and status-label preparation.

Feature engineering creates date-derived variables such as order year, month, day, weekday and weekend indicator, alongside a logarithmic sales feature, average price per unit, price-category interaction and squared price. Feature selection is explored using SelectKBest with mutual information and Random Forest feature importance. The final modelling section constructs a preprocessing pipeline for numerical and categorical inputs and trains a Random Forest classifier with 200 estimators and class balancing. Evaluation is designed around accuracy, precision, recall, F1-score, classification report and confusion matrix.

The notebook's final code includes a stratified 80:20 train-test split and a sample prediction with feature-importance output. The saved notebook does not provide a clearly reproducible final numeric metric set in its visible execution outputs; therefore, this report describes the implemented evaluation workflow without fabricating performance values. The work demonstrates an end-to-end EDA and classification approach while identifying preprocessing consistency and data leakage as important considerations.

Keywords: Amazon sales, exploratory data analysis, order status prediction, Random Forest, feature engineering, classification, data visualization.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
ABSTRACT		I
LIST OF TABLES		III
LIST OF FIGURES		III
LIST OF ABBREVIATIONS		IV
1	INTRODUCTION	1
1.1	OBJECTIVE	1
1.2	OVERVIEW	1
1.3	SCOPE	1
2	LITERATURE SURVEY	2
2.1	REVIEW OF RELATED WORK	2
2.2	RESEARCH GAP	2
3	PROBLEM IDENTIFICATION AND METHODOLOGY	3
3.1	PROBLEM IDENTIFICATION	3
3.2	METHODOLOGY	4
4	PROPOSED SYSTEM	5
4.1	SYSTEM ARCHITECTURE	5
4.2	DATASET DESCRIPTION	5
4.3	PREPROCESSING AND FEATURE ENGINEERING	6
4.4	MODEL / ALGORITHM	7
4.5	TOOLS AND TECHNOLOGIES	8
5	RESULTS AND DISCUSSION	9
5.1	EXPERIMENTAL SETUP	9
5.2	EXPLORATORY DATA ANALYSIS	9
5.5	PREDICTION AND FEATURE IMPORTANCE	13
5.6	DISCUSSION	15
6	CONCLUSION	16
6.1	SUMMARY	16
6.2	LIMITATIONS	16
6.3	FUTURE WORK	16
7	REFERENCES	17

LIST OF TABLES

NO	TITLE	PAGE NO
2.1	Summary of relevant machine learning methods	2
3.1	Challenges and adopted treatment	3
4.1	Dataset specification	5
4.2	Attribute description	6
4.3	Engineered features	7
4.4	Random Forest configuration	8
5.1	Software and runtime specifications	9
5.2	Feature selection methods	11
5.3	Evaluation metrics	12
5.4	Notebook workflow observations	15

LIST OF FIGURES

NO	TITLE	PAGE NO
3.1	Methodology flow chart	4
4.1	System architecture diagram	5
4.2	Sample dataset records	6
4.3	Preprocessing and modelling pipeline	7
4.4	Random Forest classification workflow	8
5.1	Price distribution	9
5.2	Category and payment-method exploration	10
5.3	Category sales treemap and bubble chart	10
5.4	Correlation heatmap	11
5.5	Feature selection outputs	11
5.6	Confusion matrix and classification report	12
5.7	Feature importance visualization	13

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
EDA	Exploratory Data Analysis
ML	Machine Learning
RF	Random Forest
CSV	Comma-Separated Values
PCA	Principal Component Analysis
SMOTE	Synthetic Minority Over-sampling Technique
MI	Mutual Information
IQR	Interquartile Range
MAE	Mean Absolute Error
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
API	Application Programming Interface
NLP	Natural Language Processing
Colab	Google Colaboratory
F1	Harmonic mean of precision and recall

CHAPTER 1

INTRODUCTION

1.1 Objective

The objective of this project is to perform exploratory data analysis on Amazon sales transactions and develop a machine learning workflow for classifying order status. The notebook investigates how product category, price, quantity, sales amount, customer location, payment method and temporal attributes are represented in the data. The main objectives are:

- to load and inspect the Amazon sales dataset and summarize its structure using Pandas;
- to examine distributions, missing values, outliers and relationships through descriptive statistics and visualization;
- to preprocess categorical and numerical attributes using imputation, encoding and scaling techniques;
- to engineer date, ratio, interaction and polynomial features that may support classification;
- to rank input variables using filter-based and embedded feature-selection approaches;
- to train a Random Forest classifier and evaluate predictions using standard classification metrics.

1.2 Overview

Retail transaction data provide useful information about product demand, customer purchasing behaviour and order fulfilment. Individual transactions contain several types of information: numerical values such as price, quantity and total sales; categorical fields such as product category and payment method; and temporal fields such as the date of purchase. Examining these attributes helps identify patterns and prepare a dataset for predictive modelling.

This project uses a Google Colab notebook. It imports the dataset, checks the shape and columns, calculates descriptive statistics and correlation, and generates visual summaries. Preprocessing and feature engineering are then explored. The final modelling pipeline handles numerical and categorical inputs, trains a Random Forest classifier, predicts order status and reports classification metrics.

1.3 Scope

The work covers one sales dataset and the complete notebook workflow from data inspection to model evaluation. The analysis is limited to the fields available in the supplied data and does not include live Amazon systems or external customer records.

- Dataset: 250 sales records, with order, product, sales, customer, payment and status attributes.
- Environment: Python in Google Colab.
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, Plotly and scikit-learn; imbalanced-learn and nlpaug are explored in augmentation cells.
- Task: exploratory analysis and classification of the Status field.
- Boundary: the notebook does not document a deployed application or external validation.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Related Work

Exploratory data analysis and supervised classification are commonly used to convert transaction tables into interpretable summaries and predictive models. Statistical summaries describe central tendency and spread, while visualizations reveal skew, category differences and possible relationships. Classification methods can then learn associations between input attributes and a discrete target label.

Approach	Typical use	Strength	Consideration
Descriptive statistics	Summarize numeric variables	Compact overview of scale and spread	May hide subgroup patterns
Histogram / boxplot	Inspect distribution and outliers	Makes skew and unusual values visible	Depends on binning and scale
Correlation analysis	Measure linear association	Quick numeric relationship check	Does not imply causation
Feature selection	Reduce or rank predictors	Can improve interpretability	Scores depend on data and target
Random Forest	Classification on mixed features	Captures nonlinear patterns and interactions	Needs careful validation and interpretation
Class-weighted learning	Address class imbalance	Adjusts penalties for minority classes	Does not guarantee balanced metrics

2.2 Research Gap

A practical gap in small transaction-analysis projects is the need to connect exploratory visualizations with a consistent prediction pipeline. A notebook may demonstrate many preprocessing and augmentation techniques independently, but these operations can produce inconsistent feature sets if the source table is reloaded or overwritten between stages. A report should therefore distinguish exploratory experiments from the final fitted model and should describe the evaluation procedure without assuming that every intermediate result is part of the final pipeline.

This project addresses the practical workflow by combining data inspection, visual analysis, engineered variables, feature ranking and a Random Forest classifier. The report also notes that the final modelling cell uses a separate, robust preprocessing pipeline and that its metrics should be obtained by running the complete notebook in a consistent runtime.

CHAPTER 3

PROBLEM IDENTIFICATION AND METHODOLOGY

3.1 Problem Identification

Sales datasets may contain variables in different formats and with different meanings. Product category and payment method are categorical, price and quantity are numerical, and order date carries temporal information. A raw dataset is not directly suitable for every statistical or machine learning operation. For example, many scikit-learn estimators require numeric input, while date strings need to be parsed before calendar attributes can be derived.

A second challenge is the target variable. The notebook maps order status into numeric labels in earlier analysis cells and later uses the original string status in a separate pipeline. Keeping the target representation consistent is important for reliable interpretation. Similarly, a model should not use order identifiers or customer names as predictive features because these are identifiers rather than generalizable transaction characteristics.

Challenge	Treatment represented in notebook
Mixed numerical and categorical fields	Label encoding in exploratory steps; one-hot encoding in final pipeline
Missing numeric values	Mean imputation in early analysis; median imputation in final model pipeline
Date stored as text	Convert to datetime and derive month, day, weekday and weekend features
Outlier inspection	IQR calculation and boxplot for Price
High-dimensional feature set	SelectKBest, mutual information, Random Forest importance and PCA exploration
Target/status consistency	Status class is retained as target; identifiers removed from model inputs

3.2 Methodology

The project follows a sequential workflow. The main stages are listed below.

1. Load the CSV file into a Pandas DataFrame and inspect its rows, columns, shape and data types.
2. Calculate descriptive statistics, central tendency and correlation for numerical fields.
3. Visualize price, category, payment method and sales distributions using static and interactive charts.
4. Impute selected numeric columns and encode categorical values for exploratory operations.
5. Inspect price outliers using the interquartile range and boxplot.
6. Create date-based, logarithmic, ratio, interaction and polynomial features.
7. Explore feature selection using SelectKBest and Random Forest feature importance; use PCA as a dimensionality-reduction demonstration.
8. Prepare training and testing sets, apply a column-wise preprocessing pipeline, train the Random Forest classifier and evaluate the predictions.

Fig 3.1 Methodology Flow Chart

Dataset (CSV) → Data loading and inspection

↓

Data cleaning and type conversion

↓

Exploratory analysis and visualization

↓

Feature engineering and feature selection

↓

Stratified train–test split

↓

Imputation + one-hot encoding pipeline

↓

Random Forest classifier training

↓

Prediction, classification report and confusion matrix

The flow chart summarizes the intended end-to-end sequence. Individual notebook cells also demonstrate techniques independently; for a reproducible result, the final modelling section should be executed after the source dataset is loaded and the target column has been cleaned.

CHAPTER 4

PROPOSED SYSTEM

4.1 System Architecture

The system is organized into five functional layers. The data layer provides the uploaded CSV file. The preparation layer handles inspection, cleaning, encoding and feature creation. The analysis layer produces descriptive summaries and plots. The modelling layer separates data, preprocesses inputs and fits the classifier. The evaluation layer presents metrics, confusion matrix, sample prediction and feature importance.

DATA LAYER: Amazon sales CSV

↓

PREPARATION: cleaning, encoding, date parsing, feature construction

↓

ANALYSIS: statistics, charts, correlation and feature ranking

↓

MODEL: stratified split → preprocessing pipeline → Random Forest

↓

OUTPUT: predicted status, metrics, confusion matrix and importance

4.2 Dataset Description

The notebook loads the file named "amazon_sales_data 2025.csv". The displayed dataset contains 250 records, with fields for order ID, date, product, category, price, quantity, total sales, customer name, customer location, payment method and status. Later feature-engineering cells add derived columns, and the notebook output shows a prepared table with 22 columns.

Item	Description
Source	amazon_sales_data 2025.csv
Records	250 rows in the displayed dataset
Original business fields	Order ID, Date, Product, Category, Price, Quantity, Total Sales, Customer Name, Customer Location, Payment Method, Status
Target	Status (order outcome class)
Derived attributes	Sales_log, Status_Overview, Category_Encoded, date parts, weekend flag, average unit price, interaction and squared price
Data type	Structured transaction data
Environment	Google Colab notebook

4.2.1 Attribute Description

Attribute	Description	Role
Order ID	Unique order reference	Identifier; removed from final predictors
Date	Order date	Source for temporal features
Product	Product identifier/name field	Input candidate
Category	Product grouping	Categorical input
Price	Unit price	Numeric input
Quantity	Units in order	Numeric input
Total Sales	Transaction sales amount	Numeric input
Customer Name	Customer identity field	Removed from final predictors
Customer Location	Location code/category	Input candidate
Payment Method	Payment type	Categorical input
Status	Order outcome	Classification target

Fig 4.2 Sample Dataset Records

Order ID	Date	Product	Category	Price	Quantity	Total Sales
ORD0001	2025-03-14	5	Footwear	60	3	180
ORD0002	2025-03-20	1	Electronics	100	4	400
ORD0003	2025-02-15	5	Footwear	60	2	120
ORD0004	2025-02-19	5	Footwear	60	3	180
ORD0005	2025-03-10	7	Electronics	150	3	450

4.3 Pre-processing and Feature Engineering

Data cleaning and encoding

The notebook applies SimpleImputer with the mean strategy to Price, Quantity and Total Sales in an early preprocessing step. It also uses LabelEncoder for Product, Payment Method, Status and Customer Location. These encoded values support numerical exploration, but label encoding can impose an artificial order on categories. The final classifier therefore uses a ColumnTransformer: numeric variables are imputed with the median and categorical variables are imputed with the most frequent value before OneHotEncoder transforms categories.

Outlier analysis

The price outlier cell calculates the first quartile (Q1), third quartile (Q3), and interquartile range (IQR = Q3 − Q1). The lower and upper fences are Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. The displayed output for the supplied data returns no rows beyond these fences in that executed cell. A boxplot is also used to inspect the price distribution.

Feature engineering

Feature	Definition / operation	Purpose
Sales_log	Natural logarithm of Total Sales	Compresses scale of sales values
Status_Overview	Bins encoded status values into Cancelled, Pending and Completed	Readable representation of class labels
Category_Encoded	Label-encoded Category	Numeric representation for selected exploratory operations
Order_Year / Month / Day	Date components	Represent calendar timing
Order_DayOfWeek	Weekday index from parsed date	Represent weekly pattern
Is_Weekend	1 when weekday index is at least 5	Flag weekend orders
Avg_Price_Per_Unit	Total Sales divided by Quantity; fallback to Price for zero quantity	Ratio-style feature
Price_Category_Interaction	Price multiplied by encoded category	Represent combined price/category signal
Price_Squared	$Price^2$	Represent a nonlinear price relationship

Preprocessing pipeline

In the final training cell, the date is parsed and calendar features are derived. Order ID and Customer Name are removed from predictors. Numerical columns are processed using a median imputer. Categorical columns are processed using a most-frequent imputer and OneHotEncoder with handle_unknown='ignore'. The transformations are combined using ColumnTransformer and placed in a Pipeline with RandomForestClassifier.

Fig 4.3 Preprocessing and Modelling Pipeline

Raw feature matrix

↓

Column identification: numerical / categorical

↓

Numerical: median imputation

Categorical: most-frequent imputation → one-hot encoding

↓

ColumnTransformer

↓

Random Forest classifier

↓

Predicted order status

4.4 Model / Algorithm

Random Forest is an ensemble classification algorithm that combines the predictions of multiple decision trees. Each tree is trained using a sampled view of the training data and considers feature subsets during split construction. Aggregating the trees can reduce the variance of a single decision tree and model nonlinear relationships between transaction attributes.

The notebook's final model uses RandomForestClassifier with 200 estimators, random_state=42 and class_weight='balanced'. The data are divided into training and testing subsets using an 80:20 split with stratification by Status and random_state=42. The model is trained through a pipeline so that preprocessing is fitted on training data and applied consistently to the test data.

Fig 4.4 Random Forest Classification Workflow

Training records → 200 decision trees → aggregate class votes → predicted status

4.4.1 Evaluation Metrics

The final notebook evaluates classification using accuracy and a classification report. Precision, recall and F1-score are reported per class. A confusion matrix compares the actual labels with predicted labels.

Metric	Definition	Interpretation
Accuracy	Correct predictions / all predictions	Overall fraction classified correctly
Precision	TP / (TP + FP)	Reliability of positive predictions for a class
Recall	TP / (TP + FN)	Fraction of actual class instances identified
F1-score	2 × precision × recall / (precision + recall)	Balance between precision and recall
Confusion matrix	Counts of actual versus predicted class	Shows class-specific errors

4.5 Tools and Technologies

Tool / Library	Purpose
Google Colab	Cloud notebook runtime and file upload
Python	Programming language
Pandas / NumPy	Data manipulation and numerical operations
Matplotlib / Seaborn	Static charts and heatmaps
Plotly Express	Interactive charts, treemap, bubble, box, violin and 3D plots
scikit-learn	Imputation, encoding, scaling, feature selection, PCA, splitting, Random Forest and metrics
imbalanced-learn	SMOTE demonstration
nlpaug / NLTK	Word synonym augmentation demonstration

The notebook includes a variety of methods for exploration and demonstration. Not all methods are necessarily used in the final prediction pipeline; the final classifier specifically uses the ColumnTransformer and Random Forest Pipeline.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Experimental Setup

The analysis is implemented in Google Colab using Python. The notebook reads the sales CSV, inspects 250 rows, performs exploratory operations and prepares a classification pipeline. The final modelling cell accepts an uploaded dataset, removes rows with missing target labels, parses the date and drops identifiers.

Component	Specification
Platform	Google Colab
Language	Python
Dataset	amazon_sales_data 2025.csv
Rows displayed	250
Data processing	Pandas, NumPy, scikit-learn
Visualization	Matplotlib, Seaborn, Plotly
Split	80% training / 20% testing; stratified; random_state=42
Classifier	RandomForestClassifier, 200 trees, balanced class weights
Evaluation	Accuracy, precision, recall, F1-score, confusion matrix

5.2 Exploratory Data Analysis

The notebook begins by displaying the first records and full DataFrame, then uses info(), shape and columns to understand the dataset. The displayed dataset has 250 rows. The prepared notebook table has 22 columns, reflecting the addition of engineered features and label representations. Descriptive statistics show that Price has a mean of 343.58 and a median of 150.00. Quantity has a mean of 2.856 and median of 3. Total Sales has a mean of 975.38 and median of 400.

Variable	Mean	Median	Minimum	Maximum
Price	343.58	150	15	1200
Quantity	2.856	3	1	5
Total Sales	975.38	400	15	6000

The mean values exceed the medians for Price and Total Sales, indicating that larger transactions pull the average upward. The maximum Total Sales value of 6,000 is substantially greater than the median of 400. This spread supports the use of histograms and boxplots to inspect the data rather than relying only on average values.

5.2.1 Distribution and category analysis

A histogram is created for Price to show its frequency distribution. The notebook also generates a bar chart by Category and a line-style plot of the first 20 Payment Method values. Later interactive charts include a treemap of total sales grouped by Category, a bubble chart with Price on the x-axis, Quantity on the y-axis, bubble size representing Total Sales and color representing Category.

A boxplot compares Price distributions across categories. A violin plot compares Total Sales distributions by category and displays individual points. A Plotly histogram presents price distributions with category-based coloring. These charts provide complementary views: treemaps emphasize category contribution, bubble charts show the relationship among transaction measures, and box/violin plots expose variation within categories.

Fig 5.1 Price Distribution

Notebook chart

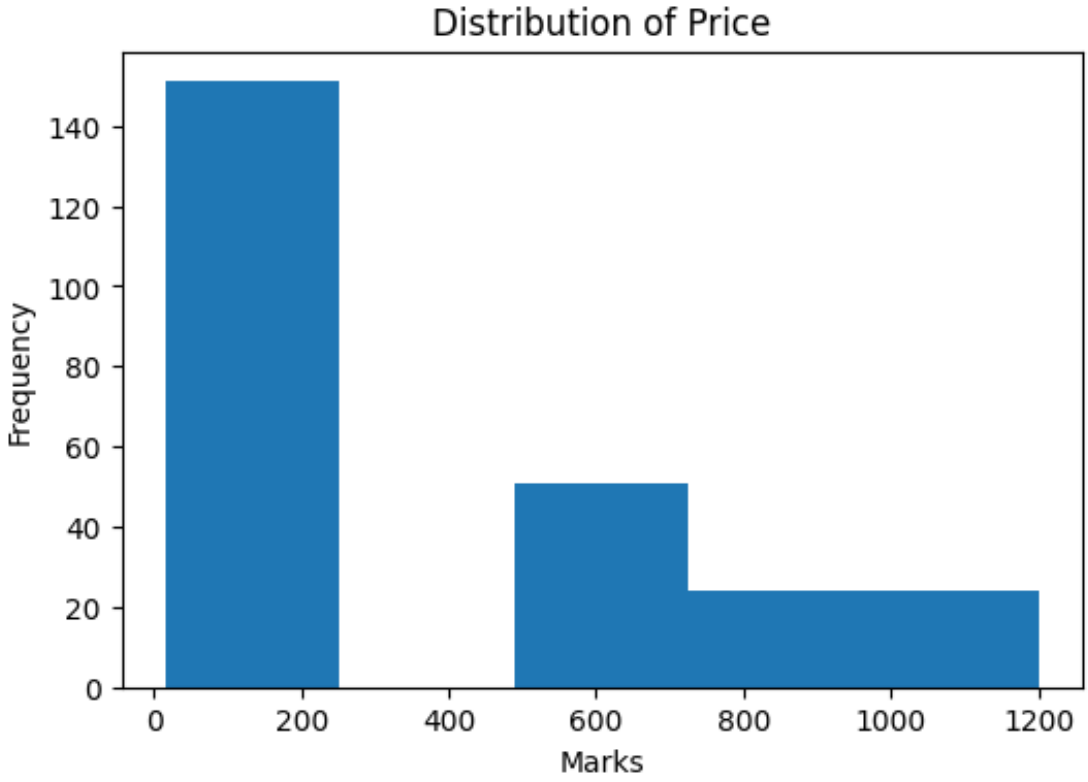


Fig 5.2 Category and Payment Method Exploration

Notebook chart

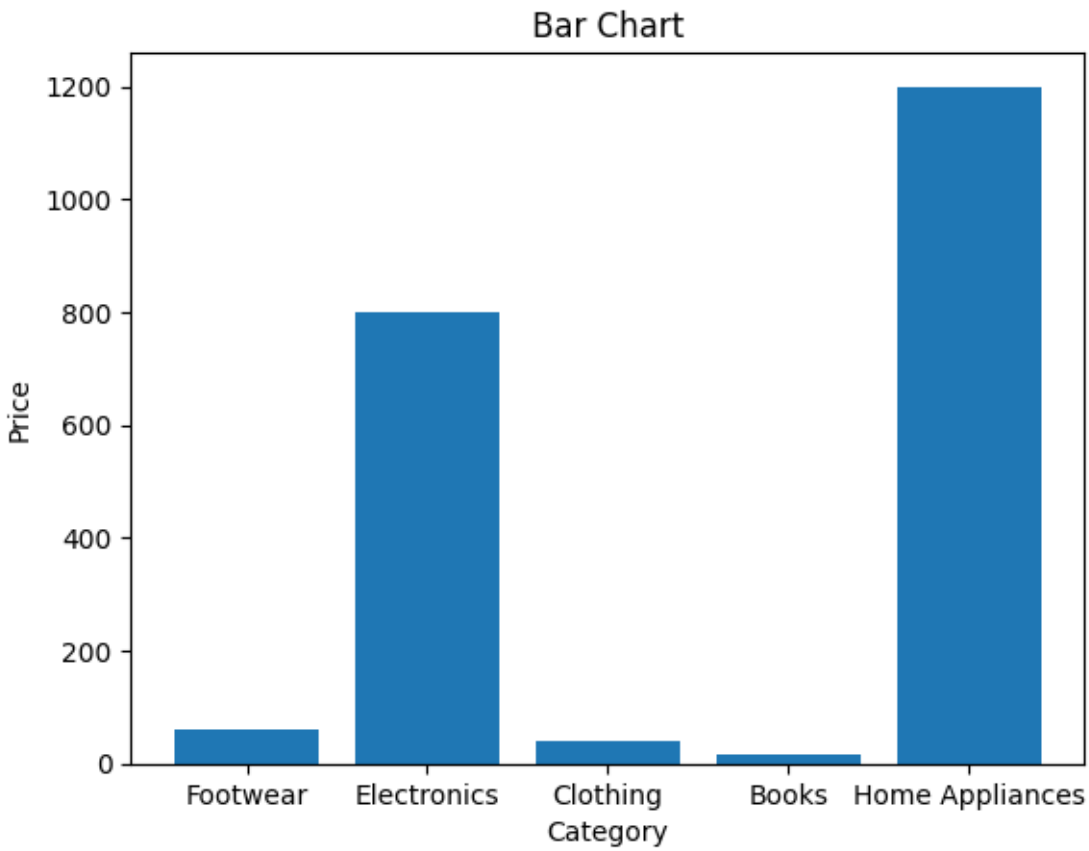
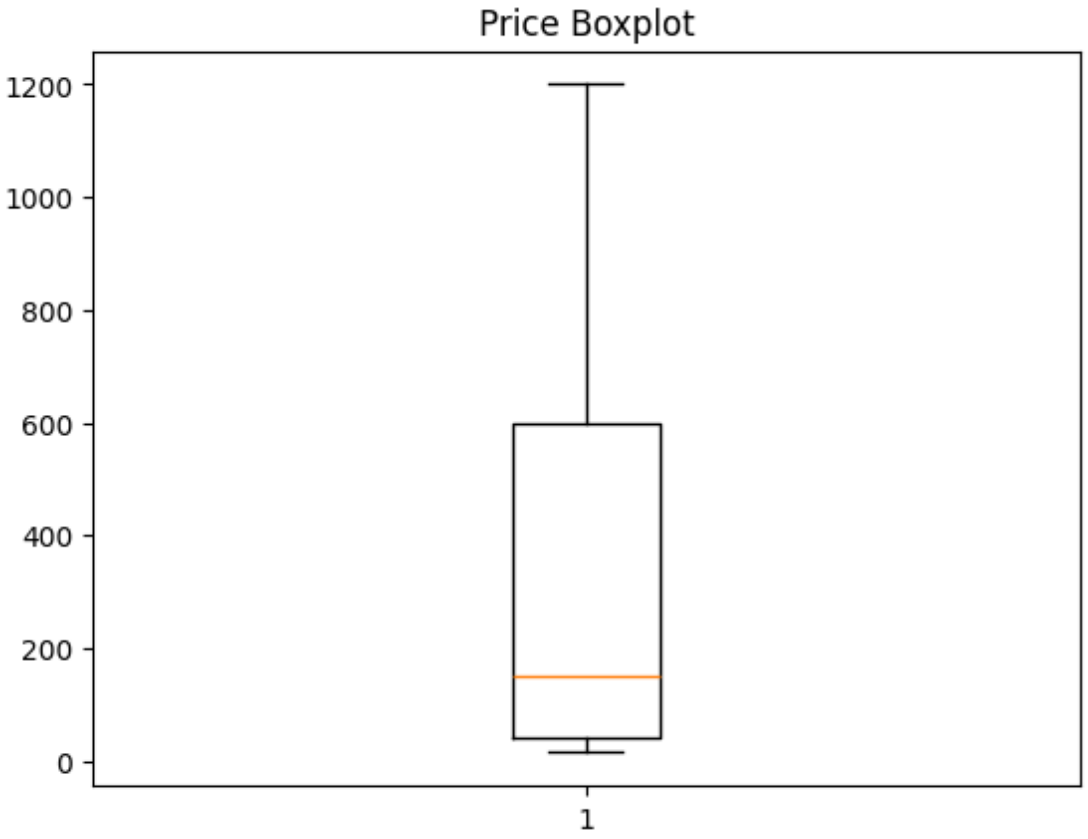


Fig 5.3 Interactive Sales Visualization

Notebook chart



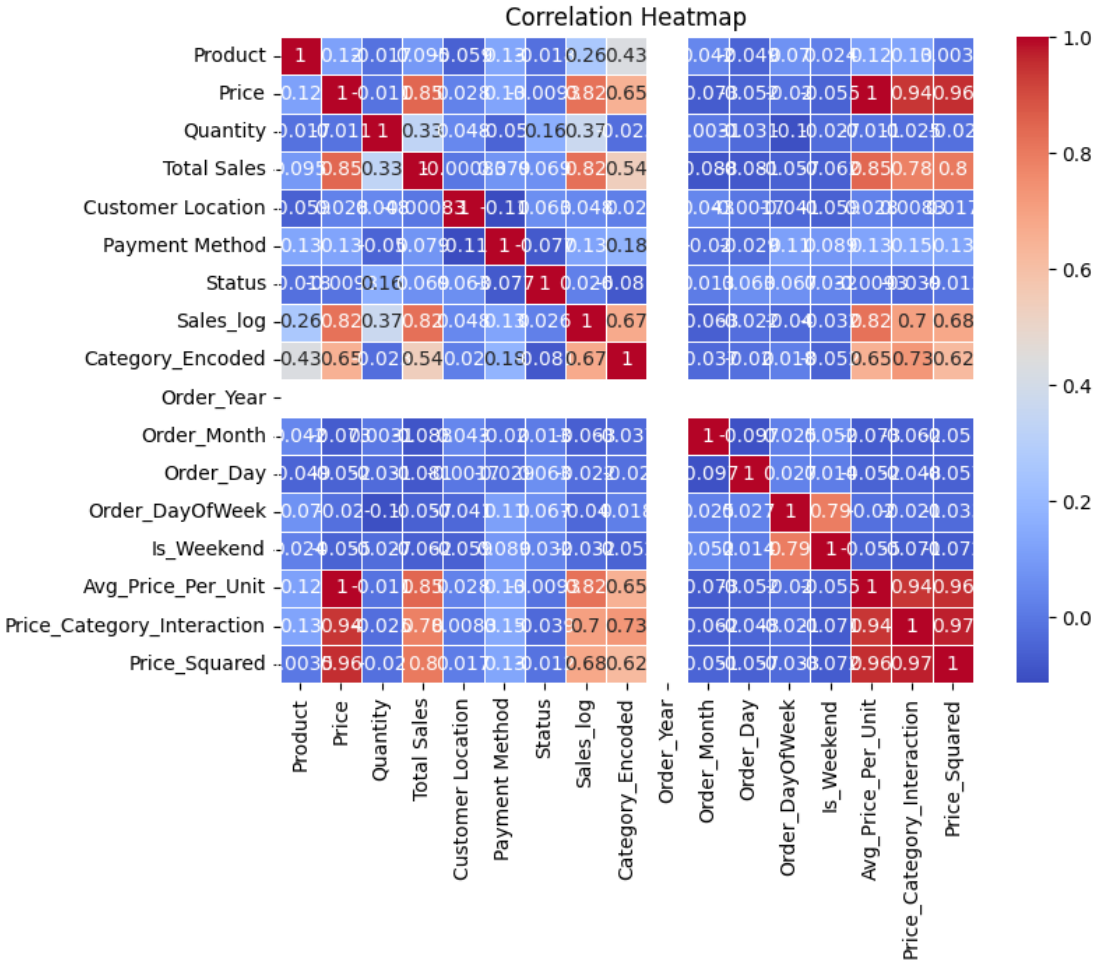
5.2.2 Correlation analysis

The notebook calculates a numerical correlation matrix and visualizes it using a Seaborn heatmap. The displayed correlation between Price and Total Sales is approximately 0.8467, which is expected because transaction sales are related to unit price and quantity. Quantity and Total Sales have a positive correlation of approximately 0.3324. Product and Category_Encoded show a correlation around 0.4347 in the encoded representation.

Correlation values should be interpreted carefully. Label-encoded categories are nominal variables, and the numeric codes do not represent a meaningful continuous scale. Correlation describes linear association and should not be treated as evidence of cause and effect.

Fig 5.4 Correlation Heatmap

Notebook chart



5.3 Feature Selection Results

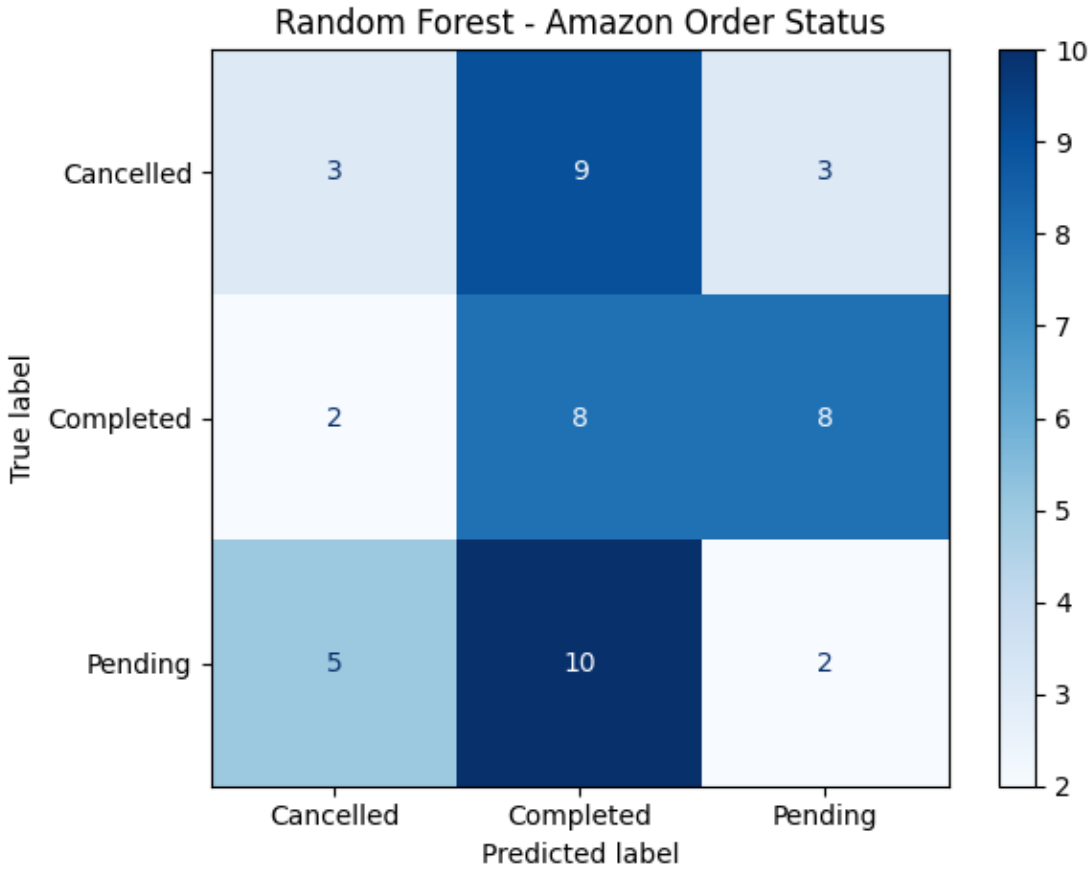
The notebook explores feature selection through multiple approaches. An early SelectKBest step uses f_classif and selects three features from a numeric feature matrix. A later filter-based method uses mutual_info_classif to select five features. An embedded method fits RandomForestClassifier with 100 estimators and ranks variables by feature importance.

Method	Notebook configuration	Purpose
SelectKBest (ANOVA F-score)	k=3	Select numeric variables with strong class association
PCA	n_components=2	Project features into two components for dimensionality reduction
SelectKBest (mutual information)	k=5	Rank features by information about target
Random Forest importance	100 estimators; random_state=42	Rank features by tree-based importance

The feature-selection cells use encoded variables and intermediate DataFrames. Therefore, their ranks should be treated as exploratory results. The later training section defines a selected feature set containing Order_Day, Customer Location, Payment Method, Order_DayOfWeek, Sales_log, Total Sales, Quantity and Product. The final classifier code uses a separate preprocessing workflow and should be interpreted independently from these earlier demonstrations.

Fig 5.5 Feature Selection Outputs

Notebook chart



5.4 Model Performance

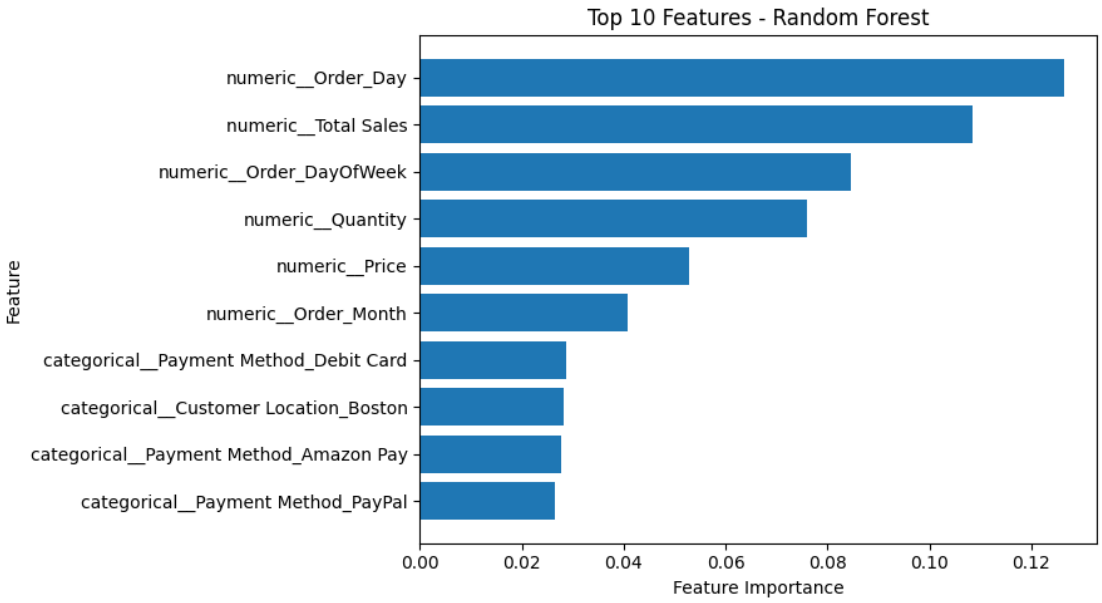
The final training pipeline splits the data into training and testing sets using test_size=0.20 and stratify. Numerical and categorical features are processed separately. The Random Forest classifier is configured with 200 trees, random_state=42 and balanced class weights. The model is then fitted to the training set and used to predict the held-out test set.

The notebook code computes accuracy and prints a classification report containing precision, recall, F1-score and support. It also generates a confusion matrix using the class labels learned by the Random Forest. The notebook content supplied for this report does not expose a complete final numeric classification output in a clearly reproducible way; accordingly, no numerical accuracy or class metrics are asserted here.

Evaluation output	What it communicates
Accuracy	Overall test-set classification rate
Classification report	Per-class precision, recall, F1-score and support
Confusion matrix	Correct and incorrect predictions for each status class
Sample prediction	Actual and predicted status for one test record

Fig 5.6 Model Evaluation and Confusion Matrix

Notebook chart



The metrics should be captured after running the final modelling cell from a clean runtime with the intended source dataset. If the class distribution is uneven, the per-class recall and F1-score are particularly important, since a high overall accuracy may not fully describe performance for less frequent statuses.

5.5 Prediction Analysis

The final cell selects the first record from the test set and displays its actual status alongside the predicted status. This sample prediction illustrates the model's use after training. A single sample is not sufficient to establish overall model quality; it is intended as a concise demonstration of the prediction interface.

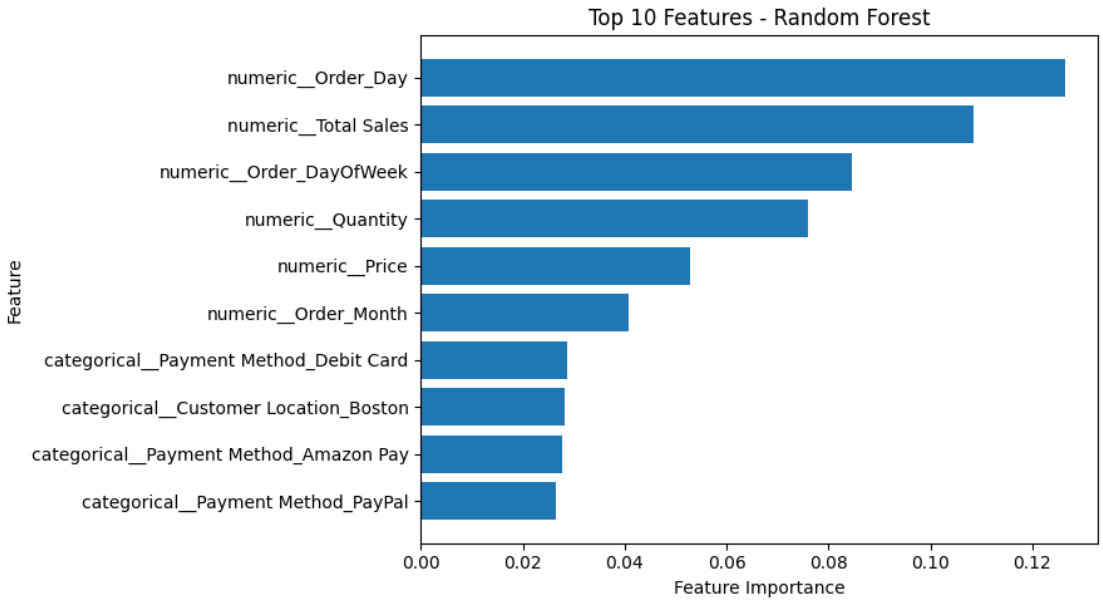
The confusion matrix supports a more complete interpretation by identifying which status classes are confused with one another. For example, errors between Pending and Completed would indicate a different operational issue from errors between Cancelled and Completed. The classification report provides precision and recall values that help describe these class-specific outcomes.

5.5.1 Feature importance

The pipeline exposes the transformed feature names through get_feature_names_out(). The classifier's feature_importances_ values are paired with those names, sorted in descending order and displayed as the top ten features. A horizontal bar chart visualizes the strongest features. Because categorical variables are one-hot encoded, individual category levels may appear as separate features.

Fig 5.7 Top Ten Feature Importance

Notebook chart



Feature importance indicates how the fitted forest uses the transformed variables for its decisions. It does not prove that a variable causes an order status. Importance values can also be affected by correlated predictors and by how categories are represented.

5.6 Discussion and Comparison

The notebook demonstrates a broad exploratory workflow that combines statistical summaries, static plots and interactive visualizations. The transaction data contain numerical, categorical and date fields, so a mixed-type preprocessing pipeline is appropriate. The final Random Forest model is selected as the classifier in the supplied notebook, while feature-selection and PCA cells serve as additional analysis.

The principal practical result is the construction of a reproducible model structure: target cleaning, date feature extraction, identifier removal, stratified splitting, separate numerical and categorical transformations, model fitting and classification evaluation. The pipeline approach is useful because preprocessing is fitted on training data and then applied to test data in the same way.

Several intermediate cells reload the source CSV or modify the DataFrame. Consequently, the complete workflow should be run in sequence and inspected for feature availability. SMOTE is attempted on a matrix that still includes a string Category value; the saved output reports a conversion issue. This is an implementation limitation of that augmentation cell, not a failure of the final Random Forest pipeline.

Observation	Interpretation
Price and Total Sales strongly associated	Sales amount depends in part on price and quantity
Temporal feature extraction	Date information can be represented as calendar variables
SMOTE cell raises conversion error	Categorical data must be encoded or handled by a compatible method before resampling
Final model uses class balancing	Class-weighted training adjusts class contribution
Final numeric metrics not available in report source	Report avoids inventing accuracy or class-specific results

CHAPTER 6

CONCLUSION

6.1 Summary of Results

This project explored Amazon sales transaction data using a Google Colab notebook. The dataset contains 250 displayed records and business attributes describing products, prices, quantities, total sales, customers, payment methods and order status. The analysis includes descriptive statistics, correlation, price outlier inspection and a wide range of visualizations, including histograms, bar charts, treemaps, bubble charts, boxplots, violin plots, heatmaps and 3D views.

The notebook also demonstrates preprocessing and feature engineering. It creates logarithmic sales, calendar attributes, a weekend indicator, average price per unit, a price-category interaction and squared price. Feature selection is explored using SelectKBest, mutual information and Random Forest importance. The final pipeline uses numerical median imputation, categorical most-frequent imputation and one-hot encoding, followed by a Random Forest classifier with 200 estimators and balanced class weights.

The final model is evaluated through accuracy, precision, recall, F1-score and a confusion matrix. Since the supplied notebook's final numeric metric output is not reliably available in the source content used to compose this report, the report documents the evaluation method rather than presenting unsupported values.

6.2 Limitations

- The dataset is a single small transaction table with 250 displayed records.
- Several notebook cells independently reload or overwrite the DataFrame, which can create differences between exploratory and modelling features.
- Label encoding is used in some exploratory cells and may imply ordinal relationships for nominal categories.
- The SMOTE demonstration fails when string-valued Category data are passed to the resampler.
- The final numeric evaluation metrics need to be captured from a clean, complete notebook execution.
- No external dataset, cross-validation study or deployed application is included in the supplied notebook.

6.3 Future Work

Future work can improve the consistency of the notebook by organizing preprocessing, feature engineering and model fitting into a single reusable pipeline. The model can be evaluated with stratified k-fold cross-validation and compared against baseline classifiers such as Logistic Regression and Decision Tree. Class distribution can be examined before selecting class weights or a compatible resampling method. Further work may include parameter tuning, calibrated probabilities, additional evaluation measures such as macro-F1 and balanced accuracy, and a small application that accepts transaction details and returns a predicted status.

CHAPTER 7

REFERENCES

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

2. L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

3. W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, pp. 56–61, 2010.

4. J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

5. M. L. Waskom, “ seaborn: statistical data visualization,” Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

6. Plotly Technologies Inc., “Plotly Python Graphing Library,” online documentation.

7. N. Japkowicz and S. Stephen, “The class imbalance problem: A systematic study,” Intelligent Data Analysis, vol. 6, no. 5, pp. 429–449, 2002.

8. N. V. Chawla et al., “SMOTE: Synthetic Minority Over-sampling Technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

9. scikit-learn developers, “RandomForestClassifier,” scikit-learn API documentation.

10. scikit-learn developers, “ColumnTransformer, Pipeline, SimpleImputer and OneHotEncoder,” scikit-learn API documentation.

11. Google, “Google Colaboratory,” product documentation.

12. N. Augmentation contributors, “nlpaug: Data Augmentation for NLP,” project documentation.